



**Tecnológico
de Monterrey**

**MA2003B Aplicación de Métodos Multivariados
en Ciencia de Datos**

Módulo 2: Relaciones de Dependencia

Profesor: Raul Gomez
Correo: rgomez@tec.mx
Oficina: A4-123A

Tabla de contenidos

I	Regresión Lineal	3
1.	Método de mínimos cuadrados	5
2.	El modelo de regresión lineal simple	15
2.1.	Estimación de parámetros por intervalos de confianza	16
2.2.	Bandas de confianza y de predicción	17
3.	Significancia y verosimilitud en el modelo de regresión lineal	19
3.1.	La función de verosimilitud	19
3.2.	Significancia del modelo y de los parámetros	20
II	Análisis Discriminante Lineal	23
4.	El modelo discriminante gaussiano	25
4.1.	El clasificador bayesiano	25
4.2.	Las funciones discriminantes lineales	26
4.3.	Las variables canónicas	27
5.	Ajuste y adecuación del modelo discriminante gaussiano	29
5.1.	Ajuste del modelo discriminante gaussiano	29
5.1.1.	El método de estimación de Fisher (FE)	29
5.1.2.	El método de máxima verosimilitud (MLE)	30
5.1.3.	Comparación de los enfoques FE y MLE	31
5.2.	Verificación de la adecuación del modelo discriminante gaussiano	31
5.2.1.	Pruebas de normalidad multivariada	31
5.2.2.	Pruebas de Homoscedasticidad	32
5.2.3.	Evaluación del rendimiento del modelo	32
5.3.	Modelos alternativos	32
6.	Método LDA paso a paso para la selección de variables	33
6.1.	Criterio de selección	33
6.2.	Flujo recomendado	34
6.3.	Ejemplo ilustrativo	34
6.3.1.	Preparación de datos	34

6.3.2. Aplicación de <code>stepclass()</code>	35
6.3.3. Ajuste del modelo final	36
6.3.4. Evaluación del modelo	37
6.3.5. Verificación de supuestos	39
6.3.6. Interpretación y reporte	42
 III Regresión Logística	 43
 7. El modelo logístico	 45
7.1. La función logística y la función logit	45
7.2. El clasificador logístico	49
 8. Ajuste y adecuación del modelo de regresión logística	 53
8.1. Ajuste del modelo de regresión logística	53
8.2. Adecuación del modelo de regresión logística	54
8.2.1. Prueba de razón de verosimilitudes	54
8.2.2. Análisis de residuos	54
8.2.3. Curvas ROC y AUC	55

Parte I

Regresión Lineal

Capítulo 1

Método de mínimos cuadrados

Supongamos que tenemos un conjunto de datos

$$((x_1, y_1), (x_2, y_2), \dots, (x_n, y_n))$$

Por ejemplo, consideremos el siguiente conjunto de datos simulado:

```
library(tidyverse)
library(krulRutils)
library(latex2exp)

set.seed(123)

n <- 50
x_data <- runif(n, min = 0, max = 10)
epsilon_data <- rnorm(n, mean = 0, sd = 1)
y_data <- x_data + epsilon_data

sample_data_tbl <- tibble(
  x = x_data,
  y = y_data
)
```

Notemos que podemos visualizar estos datos mediante un diagrama de dispersión:

```
sample_data_plot <- sample_data_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_point(
    color = c_pal("C red"),
    size = 1,
    alpha = 1
  ) +
  coord_fixed() +
  labs(
```



```
title = "Datos simulados",
x = TeX("$X_{0}$"),
y = TeX("$Y_{0}$")
) +
theme_krul()

sample_data_plot
```

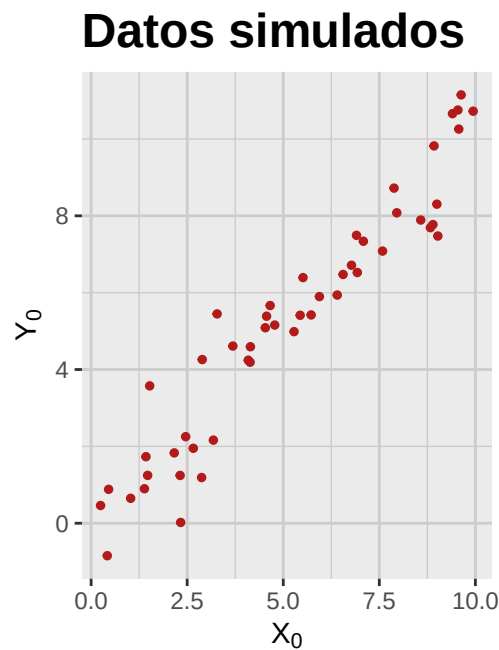


Figura 1.1: Diagrama de dispersión de los datos simulados.

Nos gustaría encontrar la línea recta que mejor se ajuste a estos datos. Hay varias formas de definir “mejor ajuste”, pero una de las más comunes es minimizar la suma de los errores al cuadrado (SSE por sus siglas en inglés) entre los valores observados y los valores correspondientes sobre la línea recta propuesta. De manera más precisa, buscamos una recta

$$Y_0 = \hat{f}(X_0) = \hat{\beta}_0 + \hat{\beta}_1 X_0$$

tal que si definimos $\hat{y}_i = \hat{f}(x_i)$ entonces minimicemos la suma

$$\text{SSE} = \sum_{i=1}^n (y_i - \hat{y}_i)^2 \quad (1.1)$$

```
fitted_data_tbl <- sample_data_tbl %>%
  mutate(
    fitted = fitted(lm(y ~ x, data = .))
  )
```

```
fitted_data_plot <- fitted_data_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +
  geom_segment(
    aes(xend = x, yend = fitted),
    color = c_pal("C green"),
    alpha = 1
  ) +
  geom_point(color = c_pal("C red"), size = 1, alpha = 1) +
  geom_point(
    aes(x = x, y = fitted),
    color = c_pal("C red"),
    size = 1,
    alpha = 1
  ) +
  coord_fixed() +
  labs(
    title = "Recta ajustada",
    x = TeX("$X_{0}$"),
    y = TeX("$Y_{0}$")
  ) +
  theme_krul()

fitted_data_plot
```

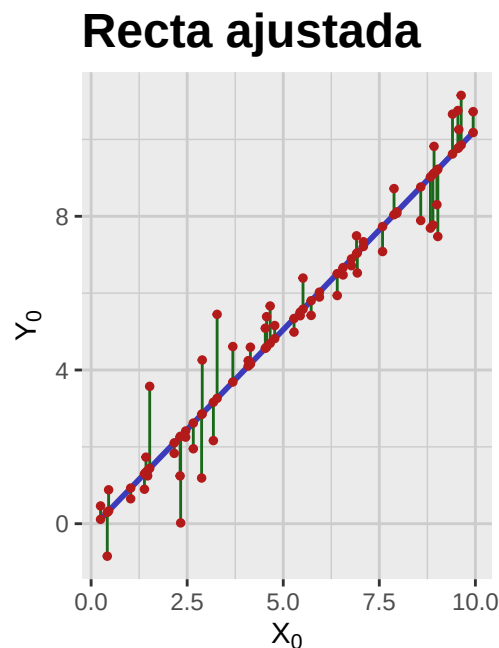


Figura 1.2: Línea recta que mejor se ajusta a los datos simulados, bajo el criterio de mínimos cuadrados. En otras palabras, esta es la línea recta que minimiza el SSE.

Para encontrar la fórmula para $\hat{\beta}_0$ y $\hat{\beta}_1$ en términos de nuestros datos, utilizaremos un poco de álgebra lineal. Empecemos por fijar los vectores

$$x = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

que llamaremos el *vector de las observaciones independientes* y el *vector de las observaciones dependientes*, respectivamente. Fijamos ahora la llamada *matriz de diseño*, dada por,

$$\Phi = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}$$

y notamos que si definimos

$$\hat{y} = \begin{bmatrix} \hat{y}_1 \\ \hat{y}_2 \\ \vdots \\ \hat{y}_n \end{bmatrix}$$

entonces $\hat{y} = \Phi \hat{\beta}$ donde

$$\hat{\beta} = \begin{bmatrix} \hat{\beta}_0 \\ \hat{\beta}_1 \end{bmatrix}$$

Notemos que para minimizar la suma de los errores al cuadrado dada en la ecuación (1.1), necesitamos encontrar la proyección ortogonal del vector y sobre el espacio generado por las columnas de la matriz Φ . Empezamos por calcular la *matriz de coeficientes*

$$D^2 = (\Phi^T \Phi)^{-1} = \begin{bmatrix} \frac{1}{n} + \frac{\bar{x}^2}{S_{xx}} & -\frac{\bar{x}}{S_{xx}} \\ -\frac{\bar{x}}{S_{xx}} & \frac{1}{S_{xx}} \end{bmatrix}$$

donde

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad \text{y} \quad S_{xx} = \sum_{i=1}^n (x_i - \bar{x})^2$$

Calculamos ahora la *matriz de estimadores de mínimos cuadrados*

$$A = D^2 \Phi^T = (\Phi^T \Phi)^{-1} \Phi^T = \begin{bmatrix} \frac{1}{n} + \frac{\bar{x}(\bar{x}-x_1)}{S_{xx}} & \cdots & \frac{1}{n} + \frac{\bar{x}(\bar{x}-x_n)}{S_{xx}} \\ \frac{x_1-\bar{x}}{S_{xx}} & \cdots & \frac{x_n-\bar{x}}{S_{xx}} \end{bmatrix}$$

y, finalmente, calculamos la *matriz sombrero*

$$H = \Phi A = \Phi (\Phi^T \Phi)^{-1} \Phi^T$$

Notemos que las entradas de la matriz H están dadas por:

$$h_{ij} = \frac{1}{n} + \frac{(x_i - \bar{x})(x_j - \bar{x})}{S_{xx}}$$

en particular,

$$h_{ii} = \frac{1}{n} + \frac{(x_i - \bar{x})^2}{S_{xx}}$$

Utilizando estas matrices, podemos calcular

$$\hat{\beta} = Ay \quad \text{y} \quad \hat{y} = Hy$$

Para ilustrar este procedimiento, consideremos el siguiente ejemplo.

Ejemplo. Consideremos el siguiente conjunto de datos:

```
set.seed(123)

n <- 5
x_data <- 1:n
epsilon_data <- rnorm(n, mean = 0, sd = n / 10)
y_data <- x_data + epsilon_data
sample_data_tbl <- tibble(
  x = x_data,
  y = y_data
)
```

Notemos que los vectores de observaciones independientes y dependientes son:

$$x = \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \\ 5 \end{bmatrix} \quad y = \begin{bmatrix} 0.72 \\ 1.88 \\ 3.78 \\ 4.04 \\ 5.06 \end{bmatrix}$$

y la matriz de diseño es:

```
Phi <- matrix(c(rep(1, n), x_data), nrow = n, ncol = 2)
```

$$\Phi = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \\ 1 & 5 \end{bmatrix}$$

Calculamos ahora la matriz de coeficientes:

```
D2 <- solve(t(Phi) %*% Phi)
```

$$D^2 = \begin{bmatrix} 1.1 & -0.3 \\ -0.3 & 0.1 \end{bmatrix}$$

y la matriz de estimadores de mínimos cuadrados:

```
A <- D2 %*% t(Phi)
```

$$A = \begin{bmatrix} 0.8 & 0.5 & 0.2 & -0.1 & -0.4 \\ -0.2 & -0.1 & 0 & 0.1 & 0.2 \end{bmatrix}$$

Finalmente, la matriz sombrero es:

```
H <- Phi %*% A
```

$$H = \begin{bmatrix} 0.6 & 0.4 & 0.2 & 0 & -0.2 \\ 0.4 & 0.3 & 0.2 & 0.1 & 0 \\ 0.2 & 0.2 & 0.2 & 0.2 & 0.2 \\ 0 & 0.1 & 0.2 & 0.3 & 0.4 \\ -0.2 & 0 & 0.2 & 0.4 & 0.6 \end{bmatrix}$$

Utilizando estas matrices, podemos rápidamente calcular los vectores $\hat{\beta}$ y $\hat{y}$:

```
beta_hat <- A %*% y_data
y_hat <- H %*% y_data
```

$$\hat{\beta} = \begin{bmatrix} -0.16 \\ 1.08 \end{bmatrix} \quad \hat{y} = \begin{bmatrix} 0.93 \\ 2.01 \\ 3.1 \\ 4.18 \\ 5.26 \end{bmatrix}$$

Notemos que en este caso la suma de los errores al cuadrado (SSE) es:

```
sse <- sum((y_data - y_hat)^2)
```

$$\text{SSE} = 0.59$$

La siguiente figura ilustra el ajuste por mínimos cuadrados para este conjunto de datos:

```
fitted_example_data_tbl <- sample_data_tbl %>%
  mutate(
    fitted = as.vector(y_hat)
  )
fitted_example_data_plot <- fitted_example_data_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +
  geom_segment(
    aes(xend = x, yend = fitted),
    color = c_pal("C green"),
    alpha = 1
  ) +
  geom_point(color = c_pal("C red"), size = 1, alpha = 1) +
```

```
geom_point(  
  aes(x = x, y = fitted),  
  color = c_pal("C red"),  
  size = 1,  
  alpha = 1  
) +  
coord_fixed() +  
labs(  
  title = "Recta ajustada",  
  x = TeX("$X_{0}$"),  
  y = TeX("$Y_{0}$")  
) +  
theme_krul()  
fitted_example_data_plot
```

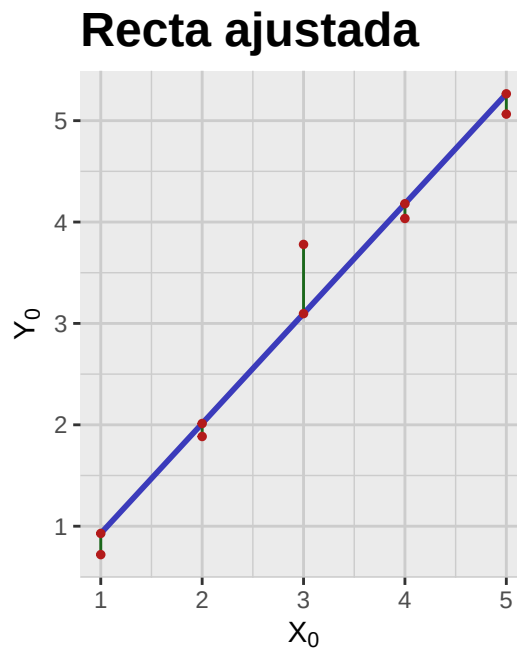


Figura 1.3: Ajuste por mínimos cuadrados para el conjunto de datos del ejemplo.

Notemos que este proceso se puede aplicar a cualquier conjunto de datos dado, sin embargo una de las aplicaciones más comunes de este método es tratar de predecir el valor de la variable Y_0 dado un valor específico de la variable X_0 que no se encuentre en el conjunto de datos original. Se sigue que para que estas predicciones sean de utilidad, es necesario que el comportamiento de nuestros datos siga una tendencia lineal, ya que en caso contrario las predicciones podrían no ser confiables. Por ejemplo, consideremos el ajuste por el método de mínimos cuadrados para los siguientes conjuntos de datos:

```
library(patchwork)

set.seed(123)

n <- 500
x_data1 <- runif(n, min = 0, max = 10)
epsilon_data1 <- rnorm(n, mean = 0, sd = 50)
y_data1 <- (x_data1)^3 + epsilon_data1

x_data2 <- runif(n, min = 0, max = 10)
epsilon_data2 <- rnorm(n, mean = 0, sd = 0.1)
y_data2 <- sin(x_data2) + epsilon_data2

x_data3 <- runif(n, min = 0, max = 10)
y_data3 <- runif(n, min = 0, max = 10)

x_data4 <- runif(n, min = 0, max = 10)
y_data4 <- mapply(function(mu, sigma) rnorm(1, mean = mu, sd = sigma),
                  mu = x_data4,
                  sigma = x_data4)

sample_data1_tbl <- tibble(
  x = x_data1,
  y = y_data1
)

sample_data2_tbl <- tibble(
  x = x_data2,
  y = y_data2
)

sample_data3_tbl <- tibble(
  x = x_data3,
  y = y_data3
)

sample_data4_tbl <- tibble(
  x = x_data4,
  y = y_data4
)

sample_data1_plot <- sample_data1_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_point(
    color = c_pal("C red"),
```

```
      size = 1,
      alpha = 1
    ) +
    geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +
    labs(
      title = "Datos cúbicos",
      x = "",
      y = ""
    ) +
    theme_krul()

sample_data2_plot <- sample_data2_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_point(
    color = c_pal("C red"),
    size = 1,
    alpha = 1
  ) +
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +
  labs(
    title = "Datos senoidales",
    x = "",
    y = ""
  ) +
  theme_krul()

sample_data3_plot <- sample_data3_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_point(
    color = c_pal("C red"),
    size = 1,
    alpha = 1
  ) +
  geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +
  labs(
    title = "Datos aleatorios",
    x = "",
    y = ""
  ) +
  theme_krul()

sample_data4_plot <- sample_data4_tbl %>%
  ggplot(aes(x = x, y = y)) +
  geom_point(
    color = c_pal("C red"),
```



```
size = 1,
alpha = 1
) +
geom_smooth(method = "lm", se = FALSE, color = c_pal("C blue")) +
labs(
  title = "Datos con varianza creciente",
  x = "",
  y = ""
) +
theme_krul()

sample_data_plot <- (sample_data1_plot + sample_data2_plot) /
  (sample_data3_plot + sample_data4_plot)

sample_data_plot
```

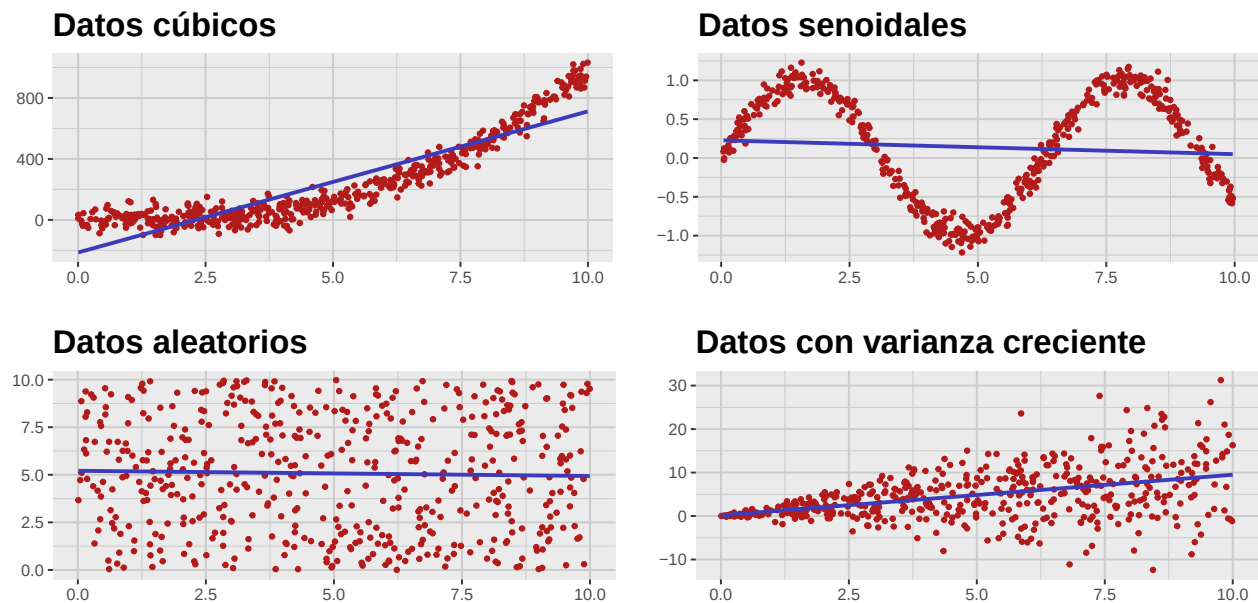


Figura 1.4: Ajuste por mínimos cuadrados para diferentes conjuntos de datos.

Es bastante claro que en estos casos el ajuste por mínimos cuadrados no es el método adecuado para realizar predicciones confiables. En general, este método solo es efectivo cuando se cumplen ciertas hipótesis que estudiaremos en las siguientes secciones, cuando introduciremos el modelo de regresión lineal desde un punto de vista estadístico.

Capítulo 2

El modelo de regresión lineal simple

En un *modelo de regresión simple*, suponemos que tenemos una *variable independiente* X_0 (también conocida como *variable explicativa* o *predictora*), y una *variable dependiente* Y_0 (también conocida como *variable de respuesta*). Suponemos además que estas variables satisfacen una relación de la forma:

$$Y_0 = f_0(X_0) + \epsilon_0$$

donde f_0 es una función desconocida que describe la relación entre ambas variables, y ϵ_0 es una variable aleatoria que representa el *error* o *ruido* en la relación entre X_0 e Y_0 . Uno de los objetivos del *aprendizaje estadístico* (también conocido como *aprendizaje automático* o *machine learning*), es tratar de aproximar la función f_0 a partir de un conjunto de datos

$$((x_1, y_1), (x_2, y_2), \dots, (x_n, y_n))$$

dado. De manera más precisa, consideremos vectores aleatorios

$$X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_n \end{bmatrix}, \quad Y = \begin{bmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{bmatrix}$$

que representan los posibles valores de las observaciones independientes y dependientes, respectivamente, en una muestra dada. En este contexto, un *método de aprendizaje* consiste de una función $\hat{F}$, que depende de los vectores aleatorios X e Y , y que cuando se evalúa en los datos observados produce una función $\hat{f}$ que aproxima a la función desconocida f_0 .

Nota 2.0.1. En muchos modelos de aprendizaje estadístico es común tomar el vector $X = x$ como fijo, en cuyo caso la función $\hat{F}$ depende únicamente del vector aleatorio Y .

Para hacer estas ideas un poco más concretas, consideremos el caso particular en el que la función desconocida f_0 es una función lineal, es decir,

$$f_0(X_0) = b_0 + b_1 X_0$$

y supondremos además que $\epsilon_0 \sim \mathcal{N}(0, \sigma^2)$ es una variable aleatoria con distribución normal de media 0 y varianza σ^2 . Este contexto se conoce como el *modelo de regresión lineal simple* y es uno de los modelos más utilizados en aprendizaje estadístico. Notemos que en este caso nuestro objetivo es encontrar una expresión que nos permita aproximar los parámetros desconocidos b_0 y b_1 en términos de los datos observados. Para ello consideraremos las expresiones que obtuvimos en la sección anterior para los coeficientes de la recta de mejor ajuste por mínimos cuadrados, pero utilizando ahora el vector aleatorio Y en lugar del vector de observaciones y . En otras palabras, definiremos

$$\Phi = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}$$

y consideraremos las matrices $D^2 = (\Phi^T \Phi)^{-1}$, $A = D^2 \Phi^T$ y $H = \Phi A$. Entonces podemos definir el *vector aleatorio de estimadores de mínimos cuadrados* como:

$$\hat{B} = AY$$

Notemos que, a diferencia de la sección anterior, en este caso $\hat{B}$ no es un vector de valores fijos, sino que es un vector aleatorio que depende linealmente del vector aleatorio Y . Además, de las propiedades de la distribución normal multivariada, sabemos que

$$\hat{B} \sim \mathcal{N}(b, \sigma^2 D^2)$$

donde

$$b = \begin{bmatrix} b_0 \\ b_1 \end{bmatrix}$$

es el vector de parámetros desconocidos. Notemos que, en particular, el vector aleatorio $\hat{B}$ es un estimador insesgado del vector de parámetros b , es decir, $E[\hat{B}] = b$. Notemos además que si evaluamos el vector aleatorio $\hat{B}$ en los datos observados $Y = y$, obtenemos el vector de coeficientes $\hat{\beta}$ de la recta de mejor ajuste por mínimos cuadrados que obtuvimos en la sección anterior. Una vez hechas estas observaciones, nuestro siguiente objetivo será construir los intervalos de confianza para los parámetros b_0 y b_1 .

2.1. Estimación de parámetros por intervalos de confianza

Empecemos por definir las cantidades

$$d_0^2 = D_{11}^2 = \frac{1}{n} + \frac{\bar{x}^2}{S_{xx}}$$

$$d_1^2 = D_{22}^2 = \frac{1}{S_{xx}}$$

donde

$$S_{xx} = \sum_{i=1}^n (x_i - \bar{x})^2$$

tal y como lo habíamos definido en la sección anterior. Entonces

$$\begin{aligned}\hat{B}_0 &\sim \mathcal{N}(b_0, \sigma^2 d_0^2) \\ \hat{B}_1 &\sim \mathcal{N}(b_1, \sigma^2 d_1^2)\end{aligned}$$

Consideremos ahora la variable aleatoria

$$\hat{S}^2 = \frac{1}{n-2} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

donde

$$\hat{Y} = \begin{bmatrix} \hat{Y}_1 \\ \hat{Y}_2 \\ \vdots \\ \hat{Y}_n \end{bmatrix} = HY$$

es el *vector aleatorio de predicciones por mínimos cuadrados*. Entonces tenemos que

$$\frac{(n-2)\hat{S}^2}{\sigma^2} \sim \chi_{n-2}^2$$

y, además, la variable aleatoria $\hat{S}^2$ es independiente de las variables aleatorias $\hat{B}_0$ y $\hat{B}_1$. Por lo tanto, podemos construir las nuevas variables aleatorias

$$\begin{aligned}T_0 &= \frac{\hat{B}_0 - b_0}{\hat{S}d_0} \sim t_{n-2} \\ T_1 &= \frac{\hat{B}_1 - b_1}{\hat{S}d_1} \sim t_{n-2}\end{aligned}$$

donde t_{n-2} denota la distribución t de Student con $n-2$ grados de libertad. Con esta información, podemos construir ahora los intervalos de confianza para los parámetros b_0 y b_1 , tal y como nos lo habíamos propuesto. Empecemos por fijar un nivel de significancia α y sea $t_{\alpha/2, n-2}$ el cuantil superior de orden $\alpha/2$ de la distribución t de Student correspondiente. Entonces, si tomamos $\hat{s}$ como el valor de la variable aleatoria $\hat{S}$ evaluada en el vector de datos observados $Y = y$, tenemos que los intervalos de confianza buscados son

$$(\hat{\beta}_0 - t_{\alpha/2, n-2} \hat{s}d_0, \hat{\beta}_0 + t_{\alpha/2, n-2} \hat{s}d_0)$$

para b_0 , y

$$(\hat{\beta}_1 - t_{\alpha/2, n-2} \hat{s}d_1, \hat{\beta}_1 + t_{\alpha/2, n-2} \hat{s}d_1)$$

para b_1 .

2.2. Bandas de confianza y de predicción

Una vez obtenidos los intervalos de confianza para los parámetros b_0 y b_1 , nos gustaría obtener ahora los intervalos de confianza para los posibles valores de la variable dependiente Y_0 en un valor

específico de la variable independiente $X_0 = x_0$. Para esto empezaremos por considerar la matriz de diseño en el punto x_0 :

$$\Phi_0 = \begin{bmatrix} 1 & x_0 \end{bmatrix}$$

y definiremos el vector aleatorio

$$\hat{Y}_0 = \Phi_0 \hat{B} = \hat{B}_0 + \hat{B}_1 x_0$$

Notemos que este vector aleatorio depende solamente de las variables $\hat{B}_0$ y $\hat{B}_1$ (las cuales, a su vez, dependen solamente de las variables $Y_1, Y_2, \dots, Y_n$), y es independiente de la variable aleatoria Y_0 . Usando nuevamente las propiedades de la distribución normal multivariada, podemos ver que

$$\hat{Y}_0 \sim \mathcal{N}(\Phi_0 b, \sigma^2 d_c^2)$$

donde

$$d_c^2 = \Phi_0 D^2 \Phi_0^T = \frac{1}{n} + \frac{(x_0 - \bar{x})^2}{S_{xx}}$$

Finalmente, como $\hat{Y}_0$ es independiente de la variable aleatoria $\hat{S}^2$, podemos construir la variable aleatoria

$$T_c = \frac{\hat{Y}_0 - \Phi_0 b}{\hat{S} d_c} \sim t_{n-2}$$

Se sigue que para construir un intervalo de confianza para el valor esperado de la variable dependiente Y_0 en el punto $X_0 = x_0$, basta con tomar el intervalo

$$(\hat{y}_0 - t_{\alpha/2, n-2} \hat{S} d_c, \hat{y}_0 + t_{\alpha/2, n-2} \hat{S} d_c)$$

donde $\hat{y}_0$ es el valor de la variable aleatoria $\hat{Y}_0$ evaluada en los datos observados $Y = y$. Notemos que este intervalo depende del punto $X_0 = x_0$ que hayamos elegido. De manera más general, podemos considerar el conjunto de todos los intervalos de confianza para todos los posibles valores de x_0 , el cual se conoce como la *banda de confianza* del modelo de regresión lineal simple.

Para terminar, notemos que como Y_0 es independiente de $\hat{Y}_0$, tenemos que

$$Y_0 - \hat{Y}_0 \sim \mathcal{N}(0, \sigma^2 d_p^2)$$

donde

$$d_p^2 = 1 + d_c^2 = 1 + \frac{1}{n} + \frac{(x_0 - \bar{x})^2}{S_{xx}}$$

Usando nuevamente la variable aleatoria $\hat{S}^2$, podemos definir ahora

$$T_p = \frac{Y_0 - \hat{Y}_0}{\hat{S} d_p} \sim t_{n-2}$$

y, de manera similar a como lo hicimos antes, podemos construir un *intervalo de predicción* para el valor de la variable dependiente Y_0 en el punto $X_0 = x_0$ como

$$(\hat{y}_0 - t_{\alpha/2, n-2} \hat{S} d_p, \hat{y}_0 + t_{\alpha/2, n-2} \hat{S} d_p)$$

Nuevamente, el conjunto de todos estos intervalos de predicción para todos los posibles valores de x_0 se conoce como la *banda de predicción* del modelo de regresión lineal simple.

Capítulo 3

Significancia y verosimilitud en el modelo de regresión lineal

Recordemos que estamos considerando un modelo de regresión lineal de la forma:

$$Y_0 = b_0 + b_1 X_{01} + b_2 X_{02} + \cdots + b_p X_{0p} + \epsilon_0$$

donde $\epsilon_0 \sim \mathcal{N}(0, \sigma^2)$. Notemos que este modelo depende de los parámetros $b_0, b_1, \dots, b_p$ y σ^2 . En este capítulo, nos enfocaremos en el estudio de estos parámetros y sus propiedades estadísticas. En particular, nos interesará conocer la significancia de los parámetros y la función de verosimilitud de los mismos.

3.1. La función de verosimilitud

Supongamos que tenemos un conjunto de datos

$$x = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix} \quad y = \begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix}$$

La función de verosimilitud del modelo de regresión lineal es una función que nos da una medida de la consistencia entre los datos dados y los parámetros considerados en el modelo. En nuestro caso, la función de verosimilitud es:

$$L(b_0, b_1, \dots, b_p, \sigma^2 | x, y) = \prod_{i=1}^n \frac{1}{\sqrt{2\pi\sigma^2}} \exp \left(-\frac{(y_i - (b_0 + b_1 x_{i1} + \cdots + b_p x_{ip}))^2}{2\sigma^2} \right)$$

que corresponde, simplemente, a la función de densidad del modelo considerado con los parámetros indicados y los datos observados. Notemos que en esta función, al contrario del capítulo anterior, estamos considerando los datos x e y como fijos, y los parámetros $b_0, b_1, \dots, b_p$ y σ^2 como variables. Nuestro siguiente objetivo será encontrar los valores de estos parámetros que maximicen la función de verosimilitud, es decir, que hagan que los datos observados sean lo más consistentes posible con el

modelo. Empezaremos por considerar la log-verosimilitud, que es simplemente el logaritmo natural de la función de verosimilitud:

$$\begin{aligned}\ell(b_0, b_1, \dots, b_p, \sigma^2 | x, y) &= \log L(b_0, b_1, \dots, b_p, \sigma^2 | x, y) \\ &= -\frac{n}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (y_i - (b_0 + b_1 x_{i1} + \dots + b_p x_{ip}))^2\end{aligned}\quad (3.1)$$

notemos que maximizar la función de verosimilitud es equivalente a maximizar la log-verosimilitud, ya que el logaritmo es una función monótonamente creciente. Por otro lado, para encontrar los valores de

$$b = \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_p \end{bmatrix}$$

que maximicen la log-verosimilitud, nos basta con minimizar la expresión:

$$\sum_{i=1}^n (y_i - (b_0 + b_1 x_{i1} + \dots + b_p x_{ip}))^2$$

Se sigue que podemos utilizar el método de los mínimos cuadrados para encontrar los valores de los parámetros que buscamos:

$$b_{\text{MLE}} = \hat{\beta} = (\Phi^T \Phi)^{-1} \Phi^T y$$

donde

$$\Phi = \begin{bmatrix} 1 & x_{11} & x_{12} & \dots & x_{1p} \\ 1 & x_{21} & x_{22} & \dots & x_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n1} & x_{n2} & \dots & x_{np} \end{bmatrix}$$

es la matriz de diseño asociada a nuestros datos. Finalmente, para encontrar el valor de σ^2 que maximice la log-verosimilitud, podemos derivar la expresión (3.1) con respecto a σ^2 y encontrar el valor que hace que la derivada sea cero. Esto nos lleva a la siguiente expresión para el estimador de máxima verosimilitud de σ^2 :

$$\sigma_{\text{MLE}}^2 = \frac{1}{n} \sum_{i=1}^n [y_i - (\hat{\beta}_0 + \hat{\beta}_1 x_{i1} + \dots + \hat{\beta}_p x_{ip})]^2 = \frac{\text{sse}}{n}$$

Desde este punto de vista, hemos obtenido una función que a cada conjunto de datos le asigna su máxima verosimilitud o, más precisamente, su máxima log-verosimilitud bajo el modelo de regresión lineal. Notemos que este valor está dado de manera explícita por:

$$\ell(\hat{\beta}, \sigma_{\text{MLE}}^2 | x, y) = -\frac{n}{2} \log(2\pi) - \frac{n}{2} \log\left(\frac{\text{sse}}{n}\right) - \frac{n}{2}$$

3.2. Significancia del modelo y de los parámetros

Nos gustaría ahora poder afirmar que los valores de la variables independientes $X_{01}, X_{02}, \dots, X_{0p}$ tienen un efecto significativo en la variable dependiente Y_0 . En particular, nos gustaría poder

afirmar que al menos uno de los parámetros $b_1, b_2, \dots, b_p$ es significativamente distinto de cero. De acuerdo al formalismo de las pruebas de hipótesis, lo que debemos hacer ahora es plantear una hipótesis nula que dice que todos los parámetros son iguales y tratar de reunir una evidencia estadísticamente significativa que nos permita rechazar esta hipótesis. Con estas consideraciones en mente, consideraremos la *suma total de cuadrados* (SST, por sus siglas en inglés) definida como:

$$SST = \sum_{i=1}^n (Y_i - \bar{Y})^2 = \|Y - \bar{Y}\mathbf{1}\|^2$$

donde

$$\bar{Y} = \frac{1}{n} \sum_{i=1}^n Y_i$$

es la media muestral de la variable dependiente y $\mathbf{1}$ es el vector columna de n entradas iguales a uno. La SST se puede interpretar como una variable aleatoria que mide la variabilidad total de los datos asociados a una observación y se puede descomponer como

$$SST = SSR + SSE$$

donde

$$SSE = \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 = \|Y - \hat{Y}\|^2$$

es la suma de los errores al cuadrado que habíamos considerado anteriormente y que corresponde a la variabilidad de los datos que no es explicada por el modelo, y

$$SSR = \sum_{i=1}^n (\hat{Y}_i - \bar{Y})^2 = \|\hat{Y} - \bar{Y}\mathbf{1}\|^2$$

es la *suma de cuadrados de la regresión* y corresponde a la variabilidad de los datos que nuestro modelo explica. Como ya habíamos mencionado anteriormente,

$$\frac{SSE}{\sigma^2} \sim \chi_{n-p-1}^2$$

Por otro lado, bajo la hipótesis nula de que todos los parámetros $b_1, b_2, \dots, b_p$ son iguales a cero, se puede demostrar que

$$\frac{SSR}{\sigma^2} \sim \chi_p^2$$

y además, que SSE y SSR son independientes. Con estas consideraciones en mente, podemos definir el estadístico de prueba

$$F = \frac{SSR/p}{SSE/(n-p-1)} \sim F_{p, n-p-1}$$

donde $F_{p, n-p-1}$ es la distribución F de Fisher con p y $n-p-1$ grados de libertad. Notemos que valores grandes de F indican que la variabilidad explicada por el modelo es grande en comparación con la variabilidad no explicada, lo cual constituye evidencia en contra de la hipótesis nula. Por lo tanto, para nuestra prueba de hipótesis, usaremos una regla de una cola por la derecha. En particular, si f es el valor de nuestro estadístico F evaluado en una observación $Y = y$, entonces el p -valor de la prueba es

$$p_F = P_F(F \geq f)$$

Si $p < \alpha$, para algún nivel de significancia α preestablecido, entonces rechazamos la hipótesis nula y concluimos que al menos uno de los parámetros $b_1, b_2, \dots, b_p$ es significativamente diferente de cero. Resumiendo, tenemos el siguiente procedimiento para evaluar la significancia del modelo de regresión lineal:

1. Plantear la hipótesis nula y la hipótesis alternativa:

H_0 : Todos los parámetros son iguales a cero, es decir $b_1 = b_2 = \dots = b_p = 0$

H_1 : Al menos uno de los parámetros $b_1, b_2, \dots, b_p$ es diferente de cero.

2. Calcular el estadístico de prueba

$$F = \frac{SSR/p}{SSE/(n-p-1)}$$

para el vector de datos observados $Y = y$.

3. Calcular el p -valor de la prueba

$$p_F = P_{F,p,n-p-1}(F \geq f)$$

donde f es el valor del estadístico F calculado en el paso anterior.

4. Si $p_F < \alpha$, para algún nivel de significancia α preestablecido, entonces rechazamos la hipótesis nula y concluimos que al menos uno de los parámetros $b_1, b_2, \dots, b_p$ es significativamente diferente de cero.

Parte II

Análisis Discriminante Lineal

Capítulo 4

El modelo discriminante gaussiano

El *análisis discriminante lineal* (LDA, por sus siglas en inglés) es un método de clasificación supervisada que permite separar dos o más clases basándose en las características más relevantes capturadas por un conjunto de datos dado. Este método de clasificación se basa en el *modelo discriminante gaussiano*, que asume que nuestra población se encuentra separada en K clases distintas (también conocidas como grupos o categorías) por una variable categórica G . El modelo también asume que tenemos un vector aleatorio continuo

$$X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_p \end{bmatrix}$$

que captura las características relevantes de los individuos en nuestra población. Una de las hipótesis más importantes de este modelo es que para cada grupo $G = i$, el vector aleatorio X sigue una distribución normal multivariada con media μ_i y matriz de covarianza Σ , es decir,

$$X|(G = i) \sim \mathcal{N}(\mu_i, \Sigma)$$

Notemos que la matriz de covarianza es la misma para todos los grupos. Esta condición es conocida como *homocedasticidad* y es uno de los supuestos fundamentales del modelo discriminante gaussiano.

4.1. El clasificador bayesiano

Recordemos que una *función de clasificación* es una función g que asigna a cada observación $X = x$ una clase $G = i$, es decir, $g(x) = i$. El objetivo del análisis discriminante lineal es encontrar una función de clasificación que minimice la *tasa de error de clasificación*, la cual está dada por

$$R(g) = E[1_{g(X) \neq G}] = P(g(X) \neq G)$$

donde $1_{g(X) \neq G}$ es una variable indicadora que toma el valor de 1 si la clasificación es incorrecta y 0 en caso contrario, $E[1_{g(X) \neq G}]$ es la esperanza de esta variable aleatoria y $P(g(X) \neq G)$ es la probabilidad de que la clasificación sea incorrecta. La solución al problema de encontrar la función de clasificación óptima está dada por el teorema de Bayes:

Teorema 4.1.1 (Teorema de Bayes para clasificación). *El clasificador que minimiza la tasa de error de clasificación es el clasificador de Bayes $\hat{g}$ que está dado por*

$$\hat{g}(x) = \arg \max_i P(G = i | X = x)$$

donde $P(G = i | X = x)$ es la probabilidad condicional de que la clase sea $G = i$ dado que la observación es $X = x$.

4.2. Las funciones discriminantes lineales

Buscamos ahora una forma más explícita para el clasificador de Bayes bajo el modelo discriminante gaussiano. Empezaremos por introducir las llamadas *probabilidades previas* (también conocidas como *probabilidades a priori* o *priors*) de cada clase, las cuales están dadas por

$$\pi_i = P(G = i)$$

Notemos que este valor es simplemente la probabilidad de que un individuo seleccionado al azar de entre nuestra población pertenezca a la clase $G = i$. También consideraremos las *probabilidades posteriores* (también conocidas como *probabilidades a posteriori* o *posteriors*) de cada clase, las cuales están dadas por

$$P(G = i | X = x)$$

En este caso, este valor representa la probabilidad de que un individuo pertenezca a la clase $G = i$ una vez que hemos observado sus características $X = x$. Para calcular este valor, recordemos que estamos suponiendo que $X | (G = i) \sim \mathcal{N}(\mu_i, \Sigma)$, por lo que la función de densidad condicional de X dado $G = i$ está dada por

$$f_i(X) = \frac{1}{(2\pi)^{p/2} |\Sigma|^{1/2}} \exp \left(-\frac{1}{2} (X - \mu_i)^T \Sigma^{-1} (X - \mu_i) \right)$$

Usando la fórmula de Bayes, podemos expresar las probabilidades posteriores en términos de las probabilidades previas y las funciones de densidad condicional:

$$P(G = i | X = x) = \frac{\pi_i f_i(x)}{\sum_j \pi_j f_j(x)}$$

Notemos que el denominador en el lado derecho de esta ecuación es el mismo para todas las clases. En particular, la probabilidad posterior $P(G = i | X = x)$ es un múltiplo positivo de la cantidad $\pi_i f_i(x)$, en símbolos,

$$P(G = i | X = x) \propto \pi_i f_i(x)$$

Se sigue que para encontrar el clasificador de Bayes, podemos concentrarnos en la expresión $\pi_i f_i(x)$. De hecho, como el logaritmo es una función monótonamente creciente, nos basta con concentrarnos en maximizar

$$\begin{aligned} \log(\pi_i f_i(x)) &= \log(\pi_i) + \log(f_i(x)) \\ &= \log(\pi_i) - \frac{p}{2} \log(2\pi) - \frac{1}{2} \log(|\Sigma|) - \frac{1}{2} (x - \mu_i)^T \Sigma^{-1} (x - \mu_i) \\ &= \log(\pi_i) - \frac{p}{2} \log(2\pi) - \frac{1}{2} \log(|\Sigma|) - \frac{1}{2} x^T \Sigma^{-1} x + \mu_i^T \Sigma^{-1} x - \frac{1}{2} \mu_i^T \Sigma^{-1} \mu_i \end{aligned}$$

Notemos que la expresión

$$-\frac{p}{2} \log(2\pi) - \frac{1}{2} \log(|\Sigma|) - \frac{1}{2} x^T \Sigma^{-1} x$$

aparece en todas nuestras clases. Por lo tanto, si definimos las *funciones discriminantes lineales* como

$$\delta_i(x) = \mu_i^T \Sigma^{-1} x - \frac{1}{2} \mu_i^T \Sigma^{-1} \mu_i + \log(\pi_i)$$

entonces tenemos que las diferencias

$$\Delta_{i,j}(x) = \log(\pi_i f_i(x)) - \log(\pi_j f_j(x)) = \delta_i(x) - \delta_j(x)$$

dependen únicamente de estas funciones. En particular, el clasificador de Bayes se puede reescribir como

$$\hat{g}(x) = \arg \max_i \delta_i(x)$$

4.3. Las variables canónicas

Además de proporcionar un método de clasificación, una de las aplicaciones más importantes del análisis discriminante lineal es la reducción de la dimensionalidad de los datos. De manera más explícita, el análisis discriminante lineal nos permite definir nuevas variables, llamadas *variables canónicas*, que capturan toda la información relevante para separar las distintas clases, pero en un espacio de menor dimensión. Para poder definir estas variables, empecemos por notar que la media total de la población está dada por

$$\mu = E[X] = \sum_i \pi_i \mu_i$$

Definimos ahora la *matriz de dispersión total* como

$$ST = E[(X - \mu)(X - \mu)^T] = \text{Var}[X]$$

Consideraremos también la *matriz de dispersión dentro de los grupos*, la cual está dada por

$$SW = E_G[\text{Var}_X[X|G]] = \sum_i \pi_i \Sigma = \Sigma$$

donde E_G representa la esperanza con respecto a la variable categórica G y $\text{Var}_X[X|G]$ representa la varianza condicional de X dado G . Finalmente, consideraremos la *matriz de dispersión entre los grupos*, la cual está dada por

$$SB = \text{Var}_G[E_X[X|G]] = \sum_i \pi_i (\mu_i - \mu)(\mu_i - \mu)^T$$

donde Var_G representa la varianza con respecto a la variable categórica G y $E_X[X|G]$ representa la esperanza condicional de X dado G . Ya con estas definiciones, podemos escribir la *fórmula de la descomposición de la varianza* que nos dice que

$$ST = SW + SB$$

Notemos ahora que si $a = [a_1, a_2, \dots, a_p]$ es una matriz de $1 \times p$, entonces podemos considerar la variable aleatoria $Y = aX$, la cual es una combinación lineal de las variables originales $X_1, X_2, \dots, X_p$. La varianza total de esta nueva variable está dada por

$$\text{Var}[Y] = a(\text{ST})a^T$$

la varianza dentro de los grupos está dada por

$$E_G[\text{Var}_Y[Y|G]] = a(\text{SW})a^T = a\Sigma a^T$$

y la varianza entre los grupos está dada por

$$\text{Var}_G[E_Y[Y|G]] = a(\text{SB})a^T$$

Para construir nuestras variable canónicas, buscamos variables que maximicen la varianza entre los grupos en relación con la varianza dentro de los grupos. En otras palabras, queremos maximizar el *cociente de Rayleigh* definido como

$$J(a) = \frac{a\text{SB}a^T}{a\text{SW}a^T}$$

De manera más precisa, el *criterio de Fisher* nos dice que las variables canónicas deben de satisfacer las siguientes condiciones:

- La primera variable canónica $Y_1 = a_1X$ es aquella que maximiza el cociente $J(a)$, es decir,

$$a_1 = \arg \max_a J(a)$$

- La j -ésima variable canónica $Y_j = a_jX$ es aquella que maximiza el cociente $J(a)$ bajo la restricción de que no esté correlacionada, dentro de los grupos, con las variables canónicas anteriores, es decir,

$$a_j = \arg \max_a J(a) \quad \text{sujeto a} \quad \text{Cov}[Y_i, Y_j|G] = 0 \quad \text{para } i < j$$

Es importante notar que el número máximo de variables canónicas que se pueden obtener es $\min(p, K - 1)$, donde p es el número de características en nuestro vector aleatorio X y K es el número de clases en nuestra población. Esto se debe a que la matriz de dispersión entre los grupos SB tiene rango máximo $r = \min(p, K - 1)$. Finalmente, se puede verificar que las funciones

$$\Delta_{i,j}(x) = \delta_i(x) - \delta_j(x)$$

solo dependen de las primeras r variables canónicas. En particular, esto implica que el clasificador de Bayes también solo depende de estas variables.

Capítulo 5

Ajuste y adecuación del modelo discriminante gaussiano

En el capítulo anterior discutimos el modelo del discriminante gaussiano desde un punto de vista teórico. También describimos como reducir la dimensionalidad de nuestro modelo proyectando los resultados de nuestras observaciones en el espacio LDA que construimos en ese capítulo y, finalmente, mostramos como podemos construir funciones discriminantes lineales que nos permiten clasificar nuestras observaciones basadas solamente en la información contenida en nuestro espacio LDA. En este capítulo estudiaremos como podemos ajustar este modelo a un conjunto de datos dado y verificar la adecuación de este ajuste. Una vez que tenemos un modelo ajustado y verificado, podemos utilizar las técnicas descritas en el capítulo anterior para reducir la dimensionalidad de nuestro conjunto de datos y construir funciones discriminantes lineales que nos permitan clasificar nuevas observaciones.

5.1. Ajuste del modelo discriminante gaussiano

Recordemos que el modelo discriminante gaussiano depende de los siguientes parámetros:

- Los vectores de medias μ_k para cada clase $k = 1, \dots, K$.
- La matriz de covarianza común Σ .
- Los pesos a priori π_k para cada clase $k = 1, \dots, K$.

Nuestro objetivo es estimar estos parámetros a partir de un conjunto de datos etiquetado $\{(x_i, g_i)\}_{i=1}^n$, donde x_i es el vector de características y g_i es la etiqueta de clase correspondiente. Existen dos enfoques principales para ajustar el modelo discriminante gaussiano: el enfoque clásico de LDA que utiliza la estimación de Fisher (FE) para la matriz de varianza común, y el enfoque basado en la estimación por máxima verosimilitud (MLE).

5.1.1. El método de estimación de Fisher (FE)

Para este método de estimación, utilizamos estimadores insesgados para los parámetros del modelo discriminante gaussiano. Las fórmulas para estimar los parámetros son las siguientes:

- Las medias de cada clase se estiman como el promedio de los vectores de características dentro de cada clase:

$$\hat{\mu}_k = \frac{1}{n_k} \sum_{i|g_i=k} x_i,$$

donde n_k es el número de observaciones en la clase k , y estamos sumando sobre todos los índices i tales que la etiqueta de clase g_i es igual a k .

- La matriz de covarianza común se estima como la media ponderada de las matrices de covarianza dentro de cada clase:

$$\hat{\Sigma}_{\text{MLE}} = \frac{1}{n - K} \sum_{k=1}^K \sum_{i|g_i=k} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T,$$

- Los pesos a priori se estiman como la proporción de observaciones en cada clase:

$$\hat{\pi}_k = \frac{n_k}{n}$$

Este enfoque es el que se utiliza comúnmente en la literatura estadística y que se implementa en R a través de la función `lda()` del paquete **MASS**.

5.1.2. El método de máxima verosimilitud (MLE)

En este método de estimación, buscamos los parámetros que maximizan la función de verosimilitud del modelo discriminante gaussiano dado el conjunto de datos etiquetado. En este caso, las fórmulas para estimar los parámetros son:

- Las medias de cada clase se estiman de la misma manera que en el método FE:

$$\hat{\mu}_k = \frac{1}{n_k} \sum_{i|g_i=k} x_i$$

- La matriz de covarianza común se estima como:

$$\hat{\Sigma}_{\text{MLE}} = \frac{1}{n} \sum_{k=1}^K \sum_{i|g_i=k} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T$$

- Los pesos a priori se estiman como:

$$\hat{\pi}_k = \frac{n_k}{n}$$

Notemos que en este caso, el estimador de la matriz de covarianza común es sesgado, pero consistente. Este enfoque es más común en la literatura de aprendizaje automático y se utiliza en implementaciones como la función `LinearDiscriminantAnalysis` del paquete **scikit-learn** en Python.

5.1.3. Comparación de los enfoques FE y MLE

Notemos que la única diferencia entre estos dos enfoques está dada en la forma en la que estimamos la matriz de covarianza común Σ . De manera más precisa, tenemos que:

$$\hat{\Sigma}_{\text{FE}} = \frac{n}{n-K} \hat{\Sigma}_{\text{MLE}}$$

En términos prácticos, esto significa que cuando utilizamos el enfoque FE para calcular las funciones discriminantes, estas funciones le dan más importancia a los valores de los priors en comparación con el enfoque MLE que le da más peso al cálculo de las medias de las clases. Cada uno de estos métodos tiene sus propias ventajas y desventajas, y la elección entre ellos depende de las características específicas del conjunto de datos y del problema en cuestión. A continuación, presentamos una tabla que resume las situaciones en las que cada enfoque puede ser más adecuado:

Contexto	FE LDA	MLE LDA
Tamaño de muestra muy pequeño	✓	
Priors muy desbalanceados	✓	
Confianza alta en los valores de los priors	✓	
Clases balanceadas		✓
Clases bien separadas		✓
Alta dimensionalidad	✓	
Muestras muy grandes	✓	✓

Tabla 5.1: Comparación de los enfoques FE y MLE para el ajuste del modelo discriminante gaussiano.

5.2. Verificación de la adecuación del modelo discriminante gaussiano

Una vez ajustado el modelo discriminante gaussiano a un conjunto de datos, es crucial verificar la adecuación de este ajuste. Esto implica evaluar si las suposiciones del modelo se cumplen y si el modelo es capaz de capturar la estructura subyacente de los datos. A continuación describiremos las pruebas y técnicas más comunes para verificar la adecuación del modelo discriminante gaussiano.

5.2.1. Pruebas de normalidad multivariada

Una de las suposiciones fundamentales del modelo discriminante gaussiano es que las observaciones dentro de cada clase siguen una distribución normal multivariada. Para verificar esta suposición, podemos utilizar pruebas estadísticas como la prueba de Mardia, la prueba de Henze-Zirkler o la prueba de Royston. Estas pruebas evalúan si los datos dentro de cada clase se ajustan a una distribución normal multivariada. Para realizar estas pruebas en R, podemos utilizar el paquete **MVN** que proporciona funciones para llevar a cabo estas pruebas de normalidad multivariada.

Otro método para evaluar la normalidad multivariada es mediante gráficos Q-Q multivariados o gráficos de dispersión de pares de variables dentro de cada clase. Estos gráficos nos permiten visualizar si los datos se ajustan a la distribución normal multivariada esperada.

5.2.2. Pruebas de Homoscedasticidad

Para evaluar si las matrices de covarianza de las diferentes clases son iguales (homoscedasticidad), podemos utilizar la prueba de Box's M. Esta prueba evalúa la hipótesis nula de que todas las matrices de covarianza son iguales contra la alternativa de que al menos una es diferente. En R, podemos utilizar la función `boxM()` del paquete `biotools` para llevar a cabo esta prueba.

Además de la prueba de Box's M, podemos utilizar gráficos de dispersión de las variables dentro de cada clase para visualizar las diferencias en las matrices de covarianza. Si las elipses de dispersión son similares en forma y tamaño, esto sugiere homoscedasticidad.

5.2.3. Evaluación del rendimiento del modelo

Finalmente, es importante evaluar el rendimiento del modelo discriminante gaussiano en términos de su capacidad para clasificar correctamente las observaciones. Podemos utilizar técnicas como la validación cruzada para estimar la tasa de error de clasificación del modelo. Además, podemos construir una matriz de confusión para visualizar el desempeño del modelo en términos de verdaderos positivos, falsos positivos, verdaderos negativos y falsos negativos.

5.3. Modelos alternativos

En caso de que las suposiciones del modelo discriminante gaussiano no se cumplan, podemos considerar modelos alternativos como el modelo discriminante cuadrático (QDA) que permite diferentes matrices de covarianza para cada clase, o modelos basados en técnicas no paramétricas como los árboles de decisión o los métodos de vecinos más cercanos (k-NN). Estos modelos pueden ser más flexibles y adaptarse mejor a conjuntos de datos que no cumplen con las suposiciones del modelo discriminante gaussiano.

Capítulo 6

Método LDA paso a paso para la selección de variables

Como vimos en el capítulo anterior, el análisis discriminante lineal (LDA) es una técnica poderosa para clasificar observaciones en grupos basándose en variables predictoras. Sin embargo, es común que no todas las variables predictoras disponibles satisfagan los supuestos del modelo o contribuyan significativamente a la discriminación entre grupos. En estos casos, disminuir el número de variables puede disminuir el ruido y mejorar la interpretabilidad y el desempeño del modelo, especialmente cuando se presentan problemas de multicolinealidad o de heterocedasticidad. En este capítulo describiremos un enfoque de selección de variables paso a paso (stepwise) para LDA utilizando el paquete `klaR` en R.

6.1. Criterio de selección

En R, el paquete `klaR` ofrece la función `stepclass()` que implementa un procedimiento de selección de variables paso a paso para modelos de clasificación, incluyendo LDA. El criterio utilizado para evaluar la inclusión o exclusión de variables es la tasa de error de clasificación estimada mediante validación cruzada (por defecto, leave-one-out). El objetivo es minimizar esta tasa de error. La función `stepclass()` permite tres estrategias de selección:

1. **Búsqueda hacia adelante** (`direction = "forward"`): comienza con un modelo nulo (sólo el intercepto) y añade variables una por una, seleccionando en cada paso la variable que más reduce la tasa de error.
2. **Búsqueda hacia atrás** (`direction = "backward"`): comienza con un modelo completo (todas las variables) y elimina variables una por una, seleccionando en cada paso la variable cuya eliminación más reduce la tasa de error.
3. **Búsqueda mixta** (`direction = "both"`): combina ambos enfoques, permitiendo añadir o eliminar variables en cada paso según cuál acción reduzca más la tasa de error.

6.2. Flujo recomendado

Para llevar a cabo la selección de variables paso a paso en LDA, usualmente se recomienda seguir el siguiente flujo:

1. **Preparación de datos:** asegúrese de que los datos estén limpios, sin valores faltantes, y que la variable de respuesta sea categórica (factor). Para las variables predictoras, todas ellas deben de ser numéricas. En este paso se puede considerar estandarizar las variables si tienen escalas muy diferentes y eliminar variables redundantes, altamente correlacionadas o con poca variabilidad. Para este proceso de eliminación previa, los diagramas de correlación y las matrices de dispersión son herramientas útiles.
2. **Aplicación de `stepclass()`:** utilice la función `stepclass()` del paquete `klaR` para realizar la selección de variables. Especifique la fórmula del modelo, los datos, el método como "lda" y la dirección de la búsqueda (hacia adelante, hacia atrás o mixta).
3. **Ajuste del modelo final:** una vez obtenida la fórmula final con las variables seleccionadas, ajuste el modelo LDA utilizando la función `lda()` del paquete `MASS`.
4. **Evaluación del modelo:** evalúe el desempeño del modelo final utilizando técnicas de validación cruzada o partición entrenamiento/prueba para estimar la tasa de error fuera de muestra. Compare los resultados con el modelo completo para verificar si la selección de variables ha mejorado el desempeño.
5. **Verificación de supuestos:** revise los supuestos del modelo LDA (normalidad multivariada y homogeneidad de matrices de covarianza) para el modelo final con las variables seleccionadas. Si los supuestos no se cumplen, considere transformaciones de variables o métodos alternativos como QDA o regularización.
6. **Interpretación y reporte:** interprete los resultados del modelo final, incluyendo las variables seleccionadas, los coeficientes discriminantes y las tasas de error. Documente el proceso de selección y justifique las decisiones tomadas durante el análisis.

6.3. Ejemplo ilustrativo

Como ejemplo ilustrativo, consideraremos el conjunto de datos `penguins` del paquete `palmerpenguins`, que contiene medidas morfológicas de tres especies de pingüinos: Adelia, Chinstrap y Gentoo. Nuestro objetivo es clasificar las especies basándonos en las medidas disponibles, seleccionando las variables más relevantes mediante el método paso a paso.

6.3.1. Preparación de datos

Empezaremos por cargar los datos y seleccionar las variables de interés, eliminando observaciones con valores faltantes.

```
library(palmerpenguins)
library(klaR)
library(MASS)
library(dplyr)
```

```
library(tidyr)

penguin_data <- penguins %>%
  dplyr::select(
    species,
    bill_length_mm,
    bill_depth_mm,
    flipper_length_mm,
    body_mass_g
  ) %>%
  drop_na() %>%
  mutate(species = droplevels(species)) %>%
  as.data.frame()
```

Nota 6.3.1. El paquete `klaR` no es compatible con tibbles, por lo que es necesario convertir el conjunto de datos a un data frame tradicional usando `as.data.frame()`.

6.3.2. Aplicación de `stepclass()`

Aplicaremos la función `stepclass()` para realizar la selección de variables utilizando las tres estrategias disponibles: hacia adelante, hacia atrás y mixta.

```
# Selección
step_fwd <- stepclass(
  species ~ .,
  data = penguin_data,
  method = "lda",
  direction = "forward"
)

step_bwd <- stepclass(
  species ~ .,
  data = penguin_data,
  method = "lda",
  direction = "backward"
)

step_bth <- stepclass(
  species ~ .,
  data = penguin_data,
  method = "lda",
  direction = "both"
)

# Fórmulas finales
form_fwd <- step_fwd$formula
```

```
form_bwd <- step_bwd$formula
form_bth <- step_bth$formula
```

Las fórmulas finales obtenidas con cada estrategia son:

```
# Convert formula to latex string
latex_formula <- function(f) {
  f_str <- deparse(f)

  # Escape underscores safely
  f_str <- gsub("_", "\\_\\_", f_str)

  # Replace '~' with '\\sim'
  f_str <- gsub("~", "\\sim", f_str)

  # Wrap cleanly in math mode
  latex <- paste0("$", f_str, "$")
  return(latex)
}

formulas_tbl <- tibble::tibble(
  Método = c("Forward", "Backward", "Both"),
  Fórmula = c(
    latex_formula(form_fwd),
    latex_formula(form_bwd),
    latex_formula(form_bth)
  )
)

knitr::kable(
  formulas_tbl,
  format = "latex",
  booktabs = TRUE,
  escape = FALSE
)
```

Método	Fórmula
Forward	$species \sim bill_length_mm + flipper_length_mm$
Backward	$species \sim bill_length_mm + bill_depth_mm + body_mass_g$
Both	$species \sim bill_length_mm + flipper_length_mm$

6.3.3. Ajuste del modelo final

Utilizamos ahora la función `lda()` para ajustar el modelo final con las variables seleccionadas por cada estrategia.

```
# Ajuste del modelo final
fit_fwd <- lda(form_fwd, data = penguin_data)
fit_bwd <- lda(form_bwd, data = penguin_data)
fit_bth <- lda(form_bth, data = penguin_data)
```

6.3.4. Evaluación del modelo

Para evaluar el desempeño de los modelos finales, calcularemos la exactitud aparente y las matrices de confusión para cada uno.

```
# Predictions
pred_fwd <- predict(fit_fwd)$class
pred_bwd <- predict(fit_bwd)$class
pred_bth <- predict(fit_bth)$class

# Accuracy
acc_fwd <- mean(pred_fwd == penguin_data$species)
acc_bwd <- mean(pred_bwd == penguin_data$species)
acc_bth <- mean(pred_bth == penguin_data$species)

# Confusion matrices
cm_fwd <- table(Verdadero = penguin_data$species, Predicción = pred_fwd)
cm_fwd <- as.data.frame.matrix(cm_fwd)
cm_fwd <- tibble::rownames_to_column(cm_fwd, var = " ")
names(cm_fwd)[1] <- "Verdadero \\ Predicción"

cm_bwd <- table(Verdadero = penguin_data$species, Predicción = pred_bwd)
cm_bwd <- as.data.frame.matrix(cm_bwd)
cm_bwd <- tibble::rownames_to_column(cm_bwd, var = " ")
names(cm_bwd)[1] <- "Verdadero \\ Predicción"

cm_bth <- table(Verdadero = penguin_data$species, Predicción = pred_bth)
cm_bth <- as.data.frame.matrix(cm_bth)
cm_bth <- tibble::rownames_to_column(cm_bth, var = " ")
names(cm_bth)[1] <- "Verdadero \\ Predicción"
```

Los resultados para las tasas de exactitud son:

```
library(kableExtra)

accuracy_tbl <- tibble::tibble(
  Método = c("Forward", "Backward", "Both"),
  Exactitud = c(acc_fwd, acc_bwd, acc_bth)
)

knitr::kable(
  accuracy_tbl,
```



```
format = "latex",
booktabs = TRUE,
digits = 4,
caption = "Exactitud de los modelos LDA por método de selección"
) %>%
kable_styling(latex_options = "hold_position")
```

Tabla 6.1: Exactitud de los modelos LDA por método de selección

Método	Exactitud
Forward	0.9561
Backward	0.9883
Both	0.9561

La matriz de confusión para el método hacia adelante es:

```
kable(
  cm_fwd,
  format = "latex",
  booktabs = TRUE,
  caption = "Matriz de confusión (Forward)"
) %>%
kable_styling(latex_options = "hold_position")
```

Tabla 6.2: Matriz de confusión (Forward)

Verdadero \ Predicción	Adelie	Chinstrap	Gentoo
Adelie	146	3	2
Chinstrap	6	59	3
Gentoo	0	1	122

La matriz de confusión para el método hacia atrás es:

```
kable(
  cm_bwd,
  format = "latex",
  booktabs = TRUE,
  caption = "Matriz de confusión (Backward)"
) %>%
kable_styling(latex_options = "hold_position")
```

Tabla 6.3: Matriz de confusión (Backward)

Verdadero \ Predicción	Adelie	Chinstrap	Gentoo
Adelie	150	1	0
Chinstrap	3	65	0
Gentoo	0	0	123

Finalmente, para el método mixto la matriz de confusión es:

```
kable(
  cm_bth,
  format = "latex",
  booktabs = TRUE,
  caption = "Matriz de confusión (Both)"
) %>%
  kable_styling(latex_options = "hold_position")
```

Tabla 6.4: Matriz de confusión (Both)

Verdadero \ Predicción	Adelie	Chinstrap	Gentoo
Adelie	146	3	2
Chinstrap	6	59	3
Gentoo	0	1	122

6.3.5. Verificación de supuestos

Es importante verificar los supuestos del modelo LDA con las variables seleccionadas. Podemos utilizar pruebas estadísticas y gráficos para evaluar la normalidad multivariada y la homogeneidad de matrices de covarianza. Empezaremos por revisar la normalidad multivariada utilizando el test de Mardia.

```
library(MVN)
# Normalidad multivariada

mardia_fwd <- mvn(
  data = penguin_data %>% dplyr::select(all_of(all.vars(form_fwd)[-1])),
  mvn_test = "mardia"
)

mardia_bwd <- mvn(
  data = penguin_data %>% dplyr::select(all_of(all.vars(form_bwd)[-1])),
  mvn_test = "mardia"
)

mardia_bth <- mvn(
```

```
data = penguin_data %>% dplyr::select(all_of(all.vars(form_bth)[-1])),
mvn_test = "mardia"
)

clean_unicode <- function(df) {
  df[] <- lapply(df, function(x) {
    # Convert to character (safe for factors/logicals)
    x <- as.character(x)
    # Remove all Unicode by transliteration
    x <- iconv(x, from = "UTF-8", to = "ASCII//TRANSLIT")
    # Drop remaining weird symbols
    x <- gsub("[^[:alnum:][:space:].,_\\-]", "", x)
    x
  })
  return(df)
}

clean_mardia_fwd <- clean_unicode(mardia_fwd$multivariate_normality)
clean_mardia_bwd <- clean_unicode(mardia_bwd$multivariate_normality)
clean_mardia_bth <- clean_unicode(mardia_bth$multivariate_normality)
```

Los resultados del test de Mardia para la selección hacia adelante son:

```
knitr::kable(
  clean_mardia_fwd,
  format = "latex",
  booktabs = TRUE,
  caption = "Test de Mardia para normalidad multivariada (Forward)"
) %>%
  kable_styling(latex_options = "hold_position")
```

Tabla 6.5: Test de Mardia para normalidad multivariada (Forward)

Test	Statistic	p.value	Method	MVN
Mardia Skewness	58.996	0.001	asymptotic	Not normal
Mardia Kurtosis	-1.555	0.12	asymptotic	Normal

Para la selección hacia atrás tenemos los siguientes resultados:

```
knitr::kable(
  clean_mardia_bwd,
  format = "latex",
  booktabs = TRUE,
  caption = "Test de Mardia para normalidad multivariada (Backward)"
) %>%
```

```
kable_styling(latex_options = "hold_position")
```

Tabla 6.6: Test de Mardia para normalidad multivariada (Backward)

Test	Statistic	p.value	Method	MVN
Mardia Skewness	105.893	0.001	asymptotic	Not normal
Mardia Kurtosis	-3.995	0.001	asymptotic	Not normal

Finalmente, para el método mixto los resultados son:

```
knitr::kable(
  clean_mardia_bth,
  format = "latex",
  booktabs = TRUE,
  caption = "Test de Mardia para normalidad multivariada (Both)"
) %>%
  kable_styling(latex_options = "hold_position")
```

Tabla 6.7: Test de Mardia para normalidad multivariada (Both)

Test	Statistic	p.value	Method	MVN
Mardia Skewness	58.996	0.001	asymptotic	Not normal
Mardia Kurtosis	-1.555	0.12	asymptotic	Normal

Para evaluar la homogeneidad de matrices de covarianza, podemos utilizar la prueba de Box.

```
library(biotools)
# Homogeneidad de matrices de covarianza
box_fwd <- boxM(
  penguin_data %>% dplyr::select(all_of(all.vars(form_fwd)[-1])),
  penguin_data$species
)

box_bwd <- boxM(
  penguin_data %>% dplyr::select(all_of(all.vars(form_bwd)[-1])),
  penguin_data$species
)

box_bth <- boxM(
  penguin_data %>% dplyr::select(all_of(all.vars(form_bth)[-1])),
  penguin_data$species
)

box_results <- tibble::tibble(
  Método = c("Forward", "Backward", "Both"),
```

```

Estadístico = c(box_fwd$statistic, box_bwd$statistic, box_bth$statistic),
`Grados de libertad` = c(
  box_fwd$parameter,
  box_bwd$parameter,
  box_bth$parameter
),
`Valor p` = c(box_fwd$p.value, box_bwd$p.value, box_bth$p.value)
)

knitr::kable(
  box_results,
  format = "latex",
  booktabs = TRUE,
  digits = 4,
  caption = "Prueba de Box para homogeneidad de matrices de covarianza"
) %>%
  kable_styling(latex_options = "hold_position")

```

Tabla 6.8: Prueba de Box para homogeneidad de matrices de covarianza

Método	Estadístico	Grados de libertad	Valor p
Forward	20.3041	6	0.0024
Backward	46.0082	12	0.0000
Both	20.3041	6	0.0024

6.3.6. Interpretación y reporte

En este ejemplo, hemos aplicado el método de selección de variables paso a paso para LDA utilizando el conjunto de datos de pingüinos. Observamos que las diferentes estrategias de selección (hacia adelante, hacia atrás y mixta) pueden conducir a diferentes conjuntos de variables seleccionadas, lo que afecta la exactitud del modelo. Una vez seleccionado el modelo final realizamos una evaluación exhaustiva del desempeño y verificamos los supuestos del modelo. Notemos que las pruebas de normalidad multivariada y homogeneidad de matrices de covarianza nos proporcionan información valiosa sobre la adecuación del modelo LDA con las variables seleccionadas. En nuestro caso, los supuestos no se cumplen completamente, sin embargo, el modelo sigue siendo útil para la clasificación. Finalmente, es crucial documentar todo el proceso de selección y evaluación para asegurar la reproducibilidad y transparencia del análisis.

Parte III

Regresión Logística

Capítulo 7

El modelo logístico

En el método de regresión logística, buscamos clasificar las observaciones dadas en dos categorías (usualmente etiquetadas como 0 y 1) basándonos en una o más variables predictoras. A diferencia de la regresión lineal, que predice valores continuos, la regresión logística predice la probabilidad de que una observación pertenezca a una categoría específica. Una vez que hemos determinado estas probabilidades, podemos utilizar el teorema de clasificación de Bayes para asignar cada observación a la categoría más probable. Notemos que el objetivo principal de este método es similar al del Análisis Discriminante Lineal (LDA), pero la regresión logística no asume que las variables predictoras sigan una distribución normal, lo que la hace más flexible en ciertos contextos. En este capítulo estudiaremos las hipótesis estadísticas asociadas con este modelo, y su implementación.

7.1. La función logística y la función logit

Recordemos que la *función logística* está definida como:

$$p = h(z) = \frac{1}{1 + e^{-z}} \quad (7.1)$$

y proporciona una biyección entre los números reales y el intervalo $(0, 1)$. Dado que las probabilidades de un evento siempre se encuentran en este intervalo, la función logística es ideal para modelar probabilidades.

```
# Definición de la función logística
logistic_function <- function(z) {
  1 / (1 + exp(-z))
}

# Ejemplo de uso de la función logística
z_data <- seq(-10, 10, length.out = 100)
p_data <- logistic_function(z_data)

logistic_tbl <- tibble(
  z = z_data,
```



```
p = p_data
)

logistic_plot <- logistic_tbl %>%
  ggplot(aes(x = z, y = p)) +
  geom_line(
    color = c_pal("C blue"),
    linewidth = 1
  ) +
  annotate(
    "text",
    x = -5, y = 0.3,
    label = TeX("$p = \\frac{1}{1 + e^{-z}}$"),
    parse = TRUE,
    size = 4,
    color = c_pal("C blue")
  ) +
  labs(
    title = "La función logística",
    x = TeX("$z$"),
    y = TeX("$p$")
  ) +
  theme_krul()

suppressWarnings(print(logistic_plot))
```

La función logística

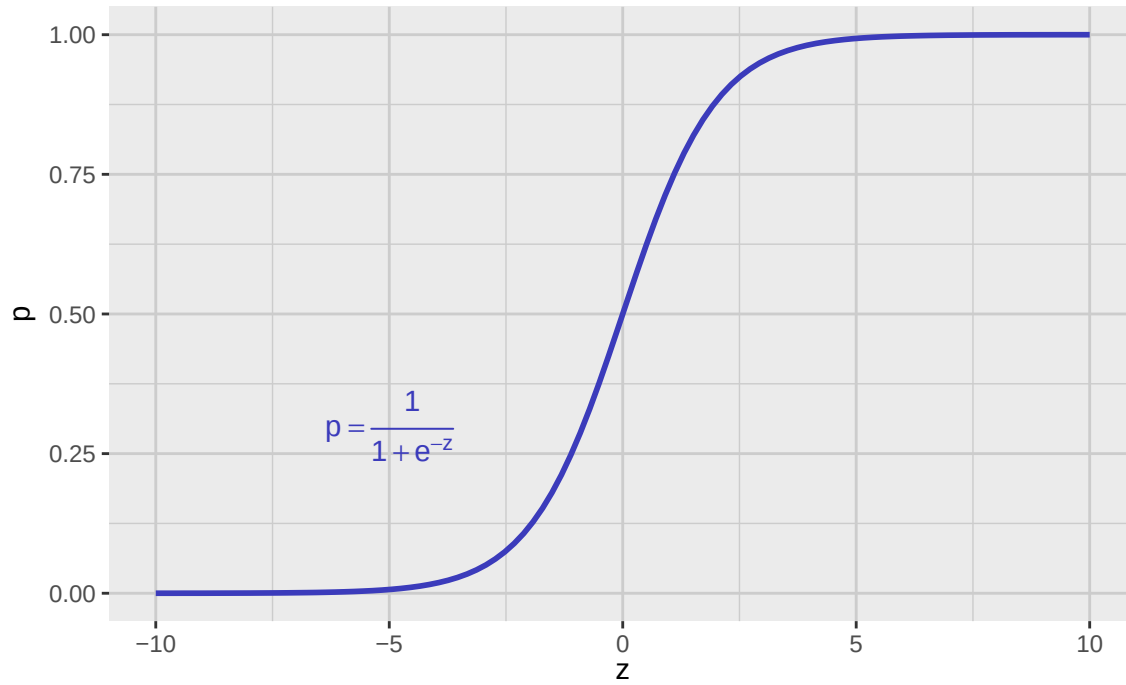


Figura 7.1: Gráfica de la función logística. Notemos que esta función establece una biyección entre los números reales y el intervalo (0, 1), lo cual es ideal para modelar probabilidades.

Notemos que la expresión dada en la ecuación (7.1) es invertible, por lo tanto podemos despejar z en términos de p . Haciendo esto, obtenemos la *función logit*, definida como:

$$z = h^{-1}(p) = \log\left(\frac{p}{1-p}\right) \quad (7.2)$$

La función logit transforma probabilidades (valores entre 0 y 1) en valores reales, lo que es útil para modelar relaciones lineales en el contexto de la regresión logística.

```
# Definición de la función logit
logit_function <- function(p) {
  log(p / (1 - p))
}

# Ejemplo de uso de la función logit
p_data_logit <- seq(0.01, 0.99, length.out = 100)
z_data_logit <- logit_function(p_data_logit)
logit_tbl <- tibble(
  p = p_data_logit,
  z = z_data_logit
)
```

```
logit_plot <- logit_tbl %>%
  ggplot(aes(x = p, y = z)) +
  geom_line(
    color = c_pal("C blue"),
    linewidth = 1
  ) +
  annotate(
    "text",
    x = 0.7, y = 2,
    label = TeX("$z = \\log\\left(\\frac{p}{1-p}\\right)$"),
    parse = TRUE,
    size = 4,
    color = c_pal("C blue")
  ) +
  labs(
    title = "La función logit",
    x = TeX("$p$"),
    y = TeX("$z$")
  ) +
  theme_krul()

suppressWarnings(print(logit_plot))
```

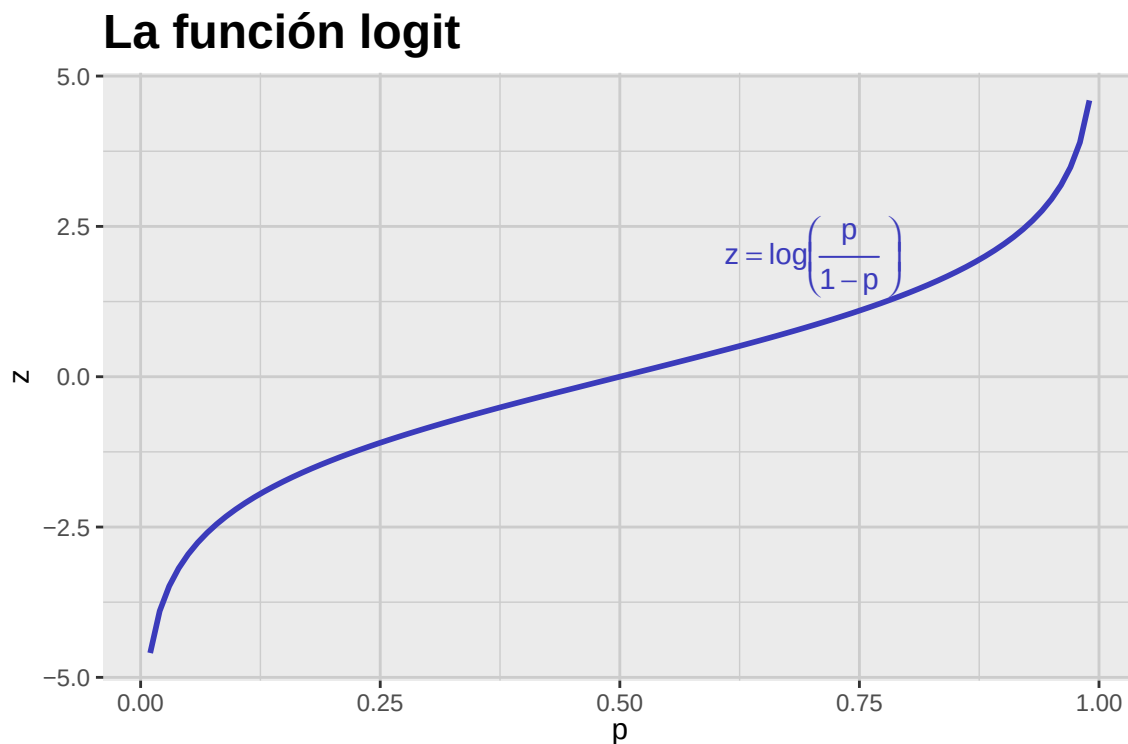


Figura 7.2: Gráfica de la función logit. Notemos que esta función transforma probabilidades (valores entre 0 y 1) en valores reales.

7.2. El clasificador logístico

Ahora supongamos que tenemos una variable de respuesta Y_0 que puede tomar solamente dos valores. (Usualmente, estos valores son 0 y 1.) Supongamos además que la relación de esta variable con un vector de variables predictoras

$$X_0 = \begin{bmatrix} X_{01} \\ X_{02} \\ \vdots \\ X_{0p} \end{bmatrix}$$

está dada por la condición

$$Y_0 | (X_0 = x_0) \sim \text{Bernoulli}(p)$$

donde $p = P(Y_0 = 1 | X_0 = x_0)$. En el modelo de regresión logística, suponemos que el logit de p depende linealmente de las variables predictoras, es decir,

$$z = \log\left(\frac{p}{1-p}\right) = b_0 + b_1 x_{01} + b_2 x_{02} + \cdots + b_p x_{0p}$$

para algunos parámetros $b_0, b_1, \dots, b_p$. Usando la función logística, podemos expresar p en términos de las variables predictoras como

$$p = h(z) = \frac{1}{1 + e^{-(b_0 + b_1 x_{01} + b_2 x_{02} + \cdots + b_p x_{0p})}}$$

Notemos que el cociente

$$\frac{p}{1-p} = e^{b_0 + b_1 x_{01} + b_2 x_{02} + \dots + b_p x_{0p}}$$

conocido como la *oportunidad* o *momio* (también conocido como *odds* en inglés) satisface la siguiente condición: por cada unidad que aumente la variable predictora X_{0j} (manteniendo las demás variables constantes), la oportunidad se multiplica por un factor de e^{b_j} . Esta propiedad hace que la regresión logística sea especialmente útil para interpretar el impacto de cada variable predictora en la probabilidad del evento de interés.

Para definir el *clasificador logístico*, fijamos un umbral de probabilidad $\tau \in (0, 1)$. Entonces, asignamos una observación al grupo 1 si la probabilidad estimada p es mayor o igual a τ , y al grupo 0 en caso contrario. En la práctica, un valor comúnmente utilizado para τ es 0.5, lo que significa que asignamos una observación al grupo 1 si la probabilidad estimada de pertenecer a ese grupo es al menos del 50 %. Sin embargo, este umbral puede ajustarse según las necesidades específicas del análisis o las consecuencias de los errores de clasificación. Notemos además que en este caso, la frontera de decisión está dada por

$$\log\left(\frac{\tau}{1-\tau}\right) = 0 = b_0 + b_1 x_{01} + b_2 x_{02} + \dots + b_p x_{0p}$$

Como ejemplo, consideremos el caso en el que tenemos dos variables predictoras ($p = 2$) y los parámetros $b_0 = 0$, $b_1 = 4$ y $b_2 = 4$. En este caso, la frontera de decisión para un umbral $\tau = 0.5$ está dada por la ecuación

$$0 + 4 \cdot x_{01} + 4 \cdot x_{02} = 0$$

$$x_{02} = -x_{01}$$

Para visualizar esta frontera, podemos simular un conjunto de observaciones y clasificar cada punto con el modelo logístico. Usaremos $b_0 = 0$, $b_1 = b_2 = 4$, $\tau = 0.5$ y generaremos 1,000 observaciones para x_1 y x_2 con distribución uniforme en $[-1, 1]$.

```
set.seed(123)

# Parámetros del modelo
b0 <- 0
b1 <- 4
b2 <- 4
tau <- 0.5

# Muestra simulada
sim_tbl <- tibble(
  x1 = runif(1000, min = -1, max = 1),
  x2 = runif(1000, min = -1, max = 1)
) %>%
  mutate(
    z = b0 + b1 * x1 + b2 * x2,
    p_hat = 1 / (1 + exp(-z)),
```

```

    grupo = if_else(runif(n()) < p_hat, "Grupo 1", "Grupo 0")
  )

# Polígonos para sombrear regiones
region_tbl <- tribble(
  ~region, ~x1, ~x2,
  "Grupo 1", -1.5, 1.5,
  "Grupo 1", 1.5, -1.5,
  "Grupo 1", 1.5, 1.5,
  "Grupo 0", -1.5, -1.5,
  "Grupo 0", 1.5, -1.5,
  "Grupo 0", -1.5, 1.5
)

ggplot() +
  geom_polygon(
    data = region_tbl,
    aes(x = x1, y = x2, fill = region, group = region),
    alpha = 0.5
  ) +
  geom_point(
    data = sim_tbl,
    aes(x = x1, y = x2, color = grupo),
    alpha = 1,
    size = 2
  ) +
  geom_abline(
    intercept = 0,
    slope = -1,
    color = "black",
    linewidth = 1
  ) +
  c_scale_color("C blue", "C green", name = "Grupo") +
  c_scale_fill("C blue", "C green", name = "Región") +
  coord_cartesian(xlim = c(-1, 1), ylim = c(-1, 1)) +
  labs(
    title = "Ejemplo del clasificador logístico",
    x = TeX("$X_{0 \\, , 1}$"),
    y = TeX("$X_{0 \\, , 2}$"),
    fill = "Región",
    color = "Observación"
  ) +
  theme_krul()

```

Ejemplo del clasificador logístico

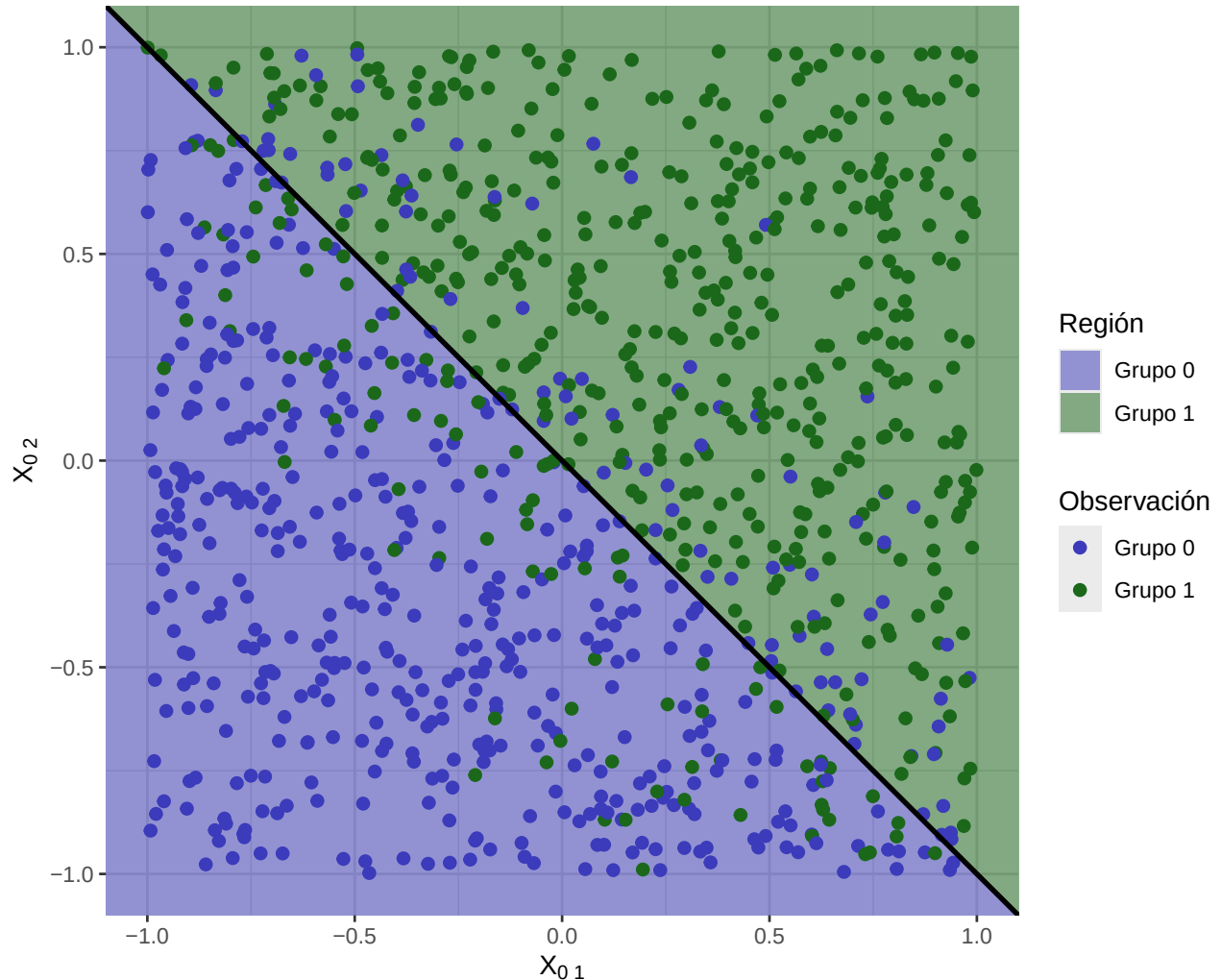


Figura 7.3: Muestra simulada con el clasificador logístico. Se usan 1,000 observaciones con X_{01} y X_{02} uniformes en $[-1, 1]$. La frontera de decisión $X_{02} = -X_{01}$ aparece en negro; y las regiones correspondientes aparecen sombreadas. Notemos que la clasificación no es perfecta, pero el número de errores decrece de manera exponencial a medida que nos alejamos de la frontera de decisión

Notemos que la clasificación no es perfecta, sin embargo el número de errores decrece de manera exponencial a medida que nos alejamos de la frontera de decisión. Si calculamos la matriz de confusión para este clasificador logístico simulado, obtenemos:

Tabla 7.1: Matriz de confusión para el clasificador logístico simulado.

Verdadero \ Predicción	Grupo 0	Grupo 1
Grupo 0	409	91
Grupo 1	93	407

Capítulo 8

Ajuste y adecuación del modelo de regresión logística

Recordemos que el modelo de regresión logística asume que la variable de respuesta Y_0 está relacionada con el vector de variables explicativas $X_0 = (X_{01}, X_{02}, \dots, X_{0p})$ a través de la siguiente distribución condicional:

$$Y_0 | (X_0 = x_0) \sim \text{Bernoulli}(p_0),$$

donde

$$\text{logit}(p_0) = b_0 + b_1 x_{01} + b_2 x_{02} + \dots + b_p x_{0p},$$

para algunos parámetros $b_0, b_1, \dots, b_p$. En este capítulo estudiaremos el problema de ajustar este modelo a un conjunto de datos y verificar su adecuación. Notemos que una vez que disponemos de un modelo ajustado y verificado, podemos utilizarlo para predecir la probabilidad de que $Y_0 = 1$ dado un valor específico de $X_0 = x_0$. A partir de esta información, podemos utilizar el clasificador de Bayes para asignar una clase a la observación $X_0 = x_0$, lo cual es una de las aplicaciones principales de la regresión logística.

8.1. Ajuste del modelo de regresión logística

Para el ajuste del modelo de regresión logística, utilizamos el método de máxima verosimilitud. Supongamos que disponemos de un conjunto de datos $\{(x_i, y_i)\}_{i=1}^n$, donde $x_i = (x_{i1}, x_{i2}, \dots, x_{ip})$ es el vector de variables explicativas para la i -ésima observación y $y_i \in \{0, 1\}$ es la variable de respuesta correspondiente. La función de verosimilitud para este conjunto de datos es:

$$L(b_0, b_1, \dots, b_p) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i},$$

donde

$$p_i = \frac{e^{b_0 + b_1 x_{i1} + \dots + b_p x_{ip}}}{1 + e^{b_0 + b_1 x_{i1} + \dots + b_p x_{ip}}}.$$

Para encontrar los estimadores de máxima verosimilitud $\hat{b}_0, \hat{b}_1, \dots, \hat{b}_p$, maximizamos la función de verosimilitud o, de manera equivalente, la log-verosimilitud:

$$\ell(b_0, b_1, \dots, b_p) = \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)].$$

Notemos que si tomamos la derivada de la log-verosimilitud con respecto a cada uno de los parámetros y la igualamos a cero, obtenemos un sistema de ecuaciones no lineales que generalmente no tiene una solución analítica cerrada. Por lo tanto, es necesario utilizar métodos numéricos para encontrar los estimadores de máxima verosimilitud. El algoritmo numérico específico depende del paquete computacional que estemos utilizando. En R, por ejemplo, podemos utilizar la función `glm()` con el argumento `family = binomial` para ajustar un modelo de regresión logística utilizando el algoritmo de Newton-Raphson. En Python, podemos utilizar la clase `LogisticRegression` del paquete `scikit-learn`, que implementa el algoritmo de optimización basado en gradiente descendente.

8.2. Adecuación del modelo de regresión logística

Una vez que hemos ajustado el modelo de regresión logística a nuestro conjunto de datos, es importante verificar la adecuación del modelo. Existen varias técnicas para evaluar la adecuación del modelo, entre las cuales destacan las siguientes:

8.2.1. Prueba de razón de verosimilitudes

Una forma común de evaluar la adecuación del modelo es mediante la prueba de razón de verosimilitudes. Esta prueba compara el modelo ajustado con un modelo nulo que no incluye ninguna variable explicativa. La estadística de la prueba se define como:

$$G = -2 [\ell(\text{modelo nulo}) - \ell(\text{modelo ajustado})],$$

donde $\ell(\text{modelo nulo})$ es la log-verosimilitud del modelo nulo y $\ell(\text{modelo ajustado})$ es la log-verosimilitud del modelo ajustado. Bajo la hipótesis nula de que el modelo ajustado no mejora significativamente el ajuste en comparación con el modelo nulo, la estadística G sigue una distribución chi-cuadrado con p grados de libertad, donde p es el número de variables explicativas en el modelo ajustado. Si el valor p asociado a la estadística G es menor que un nivel de significancia predefinido (por ejemplo, 0.05), rechazamos la hipótesis nula y concluimos que el modelo ajustado proporciona un mejor ajuste que el modelo nulo.

8.2.2. Análisis de residuos

Otra técnica para evaluar la adecuación del modelo es el análisis de residuos. En el contexto de la regresión logística, los residuos pueden definirse como la diferencia entre las observaciones reales y las probabilidades predichas por el modelo. Podemos analizar los residuos para detectar patrones sistemáticos que puedan indicar un mal ajuste del modelo. Por ejemplo, podemos graficar los residuos contra las probabilidades predichas o contra cada una de las variables explicativas para identificar posibles tendencias o heterocedasticidad. Además, podemos utilizar pruebas estadísticas, como la prueba de Hosmer-Lemeshow, para evaluar la adecuación del modelo basándonos en los residuos.

8.2.3. Curvas ROC y AUC

Las curvas ROC (Receiver Operating Characteristic) y el área bajo la curva (AUC, Area Under the Curve) son herramientas útiles para evaluar el desempeño del modelo de regresión logística en términos de clasificación. La curva ROC representa la relación entre la tasa de verdaderos positivos (sensibilidad) y la tasa de falsos positivos ($1 - \text{especificidad}$) para diferentes umbrales de clasificación. El AUC proporciona una medida cuantitativa del desempeño del modelo, donde un valor de AUC cercano a 1 indica un buen desempeño, mientras que un valor cercano a 0.5 indica un desempeño similar al azar. Podemos utilizar estas métricas para comparar diferentes modelos de regresión logística y seleccionar el que mejor se ajuste a nuestros datos.

